

CampusEats — Service Design

Assignment 2 — Design Document

This document contains the capability list, service contracts, the complete **placeOrder** operation specification, and service validation.

Service decomposition used throughout this document

User Service — Gaurav Vyas; Menu Service — MS Suhan; Cart Service — Shubham Parmar; Order Service — Manish Saxena; Payment Service — Kunal Roy.

Data ownership is separated by service: Users; Menu/Food Items; Cart/Cart Items; Orders/Order Items; Payments.

Task 1 — Capabilities

The following are the distinct jobs/capabilities identified for CampusEats. They are grouped later under the services responsible for them.

No.	Capability	Responsible Service
1	Register User	User Service
2	Login User	User Service
3	Browse Menu	Menu Service
4	Manage Menu	Menu Service
5	Add Item to Cart	Cart Service
6	Remove Item from Cart	Cart Service
7	View Cart	Cart Service
8	Place Order	Order Service
9	Cancel Order	Order Service
10	Manage Order	Order Service
11	Track Order	Order Service
12	View Order History	Order Service
13	Make Payment	Payment Service

Service boundaries and owned data

User Service owns Users. Menu Service owns Menu and Food Items. Cart Service owns Cart and Cart Items. Order Service owns Orders and Order Items. Payment Service owns Payments. No table/data entity is owned by more than one service.

Task 3 — Service Contracts

The contracts below define the externally visible operations used by the service design. Internal database details are not exposed to callers.

Service	Operation	Inputs	Success Output	Error Cases
User Service	registerUser	name, email, password	userId, created status	invalid input; duplicate email
User Service	loginUser	email, password	authentication result / userId	invalid credentials; user not found
Menu Service	browseMenu	optional category/filter	list of menu items	invalid filter; service unavailable
Menu Service	checkItem	foodItemId	item existence and availability	item not found
Menu Service	manageMenu	menu/food-item data and action	updated menu status	invalid item; validation failure
Cart Service	addItemToCart	userId, foodItemId, quantity	updated cart	invalid item; invalid quantity
Cart Service	removeItemFromCart	userId, foodItemId	updated cart	cart/item not found
Cart Service	getCart	userId	cart with cart items	cart not found; user invalid
Order Service	placeOrder	userId, cartId, payment details	orderId, order status, payment result	cart invalid/empty; item unavailable; payment failure
Order Service	cancelOrder	orderId, userId	cancellation status	order not found; cancellation not allowed
Order Service	manageOrder	orderId, action/status	updated order status	order not found; invalid status transition
Order Service	trackOrder	orderId, userId	current order status	order not found; unauthorized access
Order Service	viewOrderHistory	userId	list of previous orders	user not found; service unavailable
Payment Service	createPayment	orderId, amount, payment method	paymentId, payment status	invalid amount; payment failure
Payment Service	getPaymentStatus	paymentId	payment status	payment not found

Task 4 — Full Specification of placeOrder

Operation: placeOrder

Purpose: Convert a valid user's cart into a new order, verify that the requested items are still available, request payment, and return the created order and payment outcome without exposing internal database details.

Inputs: userId, cartId, paymentMethod/payment details. The cart must belong to the user.

Successful output: orderId, order status, paymentId (when applicable), and payment status. The caller does not receive internal table structures, database connections, or implementation details.

Preconditions

The user is valid; the cart exists and belongs to that user; the cart is not empty; each requested food item can be validated by Menu Service; quantities are valid; and the required payment information is present.

Processing / Hidden Internal Steps

1. Obtain the cart through Cart Service using **getCart()**.
2. Validate each cart item through Menu Service using **checkItem()**.
3. Calculate the order total using the current valid item information.
4. Create the order and order-item records inside Order Service's own database.
5. Request payment through Payment Service using **createPayment()**.
6. If payment succeeds, finalize the order status and return the result.
7. If payment fails, do not expose internal database state; return a payment failure and leave the order in a clearly defined non-completed state.

Error Cases

Invalid user; cart not found; cart does not belong to the user; empty cart; food item unavailable/not found; invalid quantity; payment failure; invalid payment data; duplicate/repeated request where applicable; or dependent service unavailable.

Postconditions

On success, a unique order exists in Order Service and has a successful payment result. On failure before successful completion, the caller receives a defined error and no completed order is returned.

What is hidden from callers

Database schema, table names, SQL queries, transaction implementation, internal IDs other than the public order/payment identifiers, service credentials, connection details, and internal retry/locking mechanisms.

Task 6 — Service Validation

Each service is checked against the five required Service properties: reachable, self-contained, has a contract, independent, and loosely coupled.

Service	Reachable	Self-contained	Has a contract	Independent	Loosely coupled	Result / Fix
User Service	Yes	Yes	Yes	Yes	Yes	No failure. Owns Users and exposes user operations.
Menu Service	Yes	Yes	Yes	Yes	Yes	No failure. Owns Menu/Food Items; callers use its contract.
Cart Service	Yes	Yes	Yes	Yes	Yes	No failure. Owns Cart/Cart Items and calls Menu through <code>checkItem()</code> .
Order Service	Yes	Yes	Yes	Yes	Yes	No failure. Owns Orders/Order Items and uses Cart/Payment contracts.
Payment Service	Yes	Yes	Yes	Yes	Yes	No failure. Owns Payments and exposes payment operations through its contract.

Validation conclusion: All five services satisfy the five properties in the current design. No service currently fails validation, so no corrective change is required. If implementation introduces a shared database, direct database access, or undocumented endpoints, the affected service would fail the self-contained, independent, or loosely-coupled checks and should be fixed by restoring service-owned data and contract-based communication.